Arduino Pan–Tilt Distance Meter

Wiring diagram, components and working guide

Based on the Arduino Uno sketch supplied by Azam Sahriar • 20 October 2026

This project combines manual pan and tilt control, continuous ultrasonic distance measurement, a pushbutton-operated laser pointer and a 16×2 status display. Moving the joystick aims the platform; the sensor measures distance continuously.

Hardware assumptions: Arduino Uno R3 / ATmega328P, HC-SR04 sensor, 5 V 28BYJ-48 motor with ULN2003 board, a hobby servo rated for the chosen supply, a parallel HD44780-compatible 16×2 LCD, and a laser driver with a control input compatible with the Uno's 5 V output.

1. System connection diagram

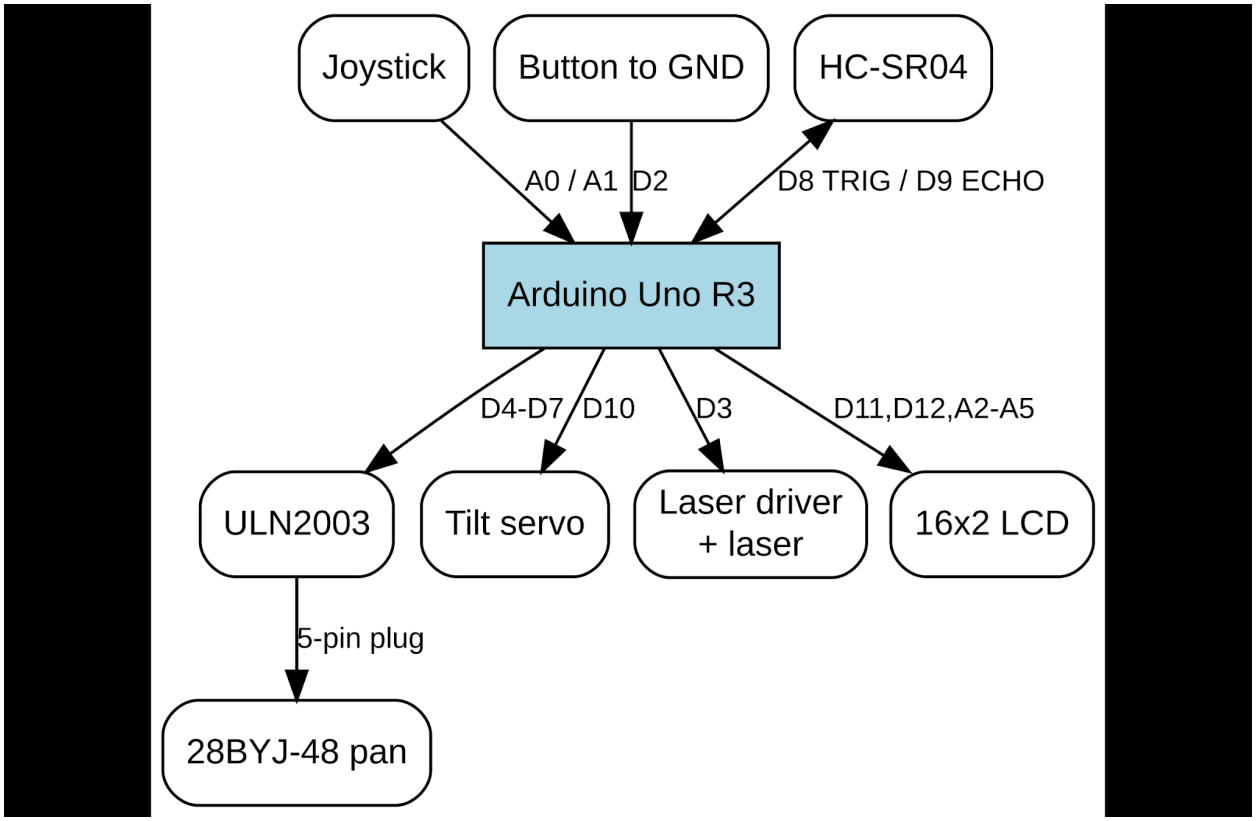


Figure 1. Signal connections taken from the supplied sketch. Power and common-ground connections are specified in Section 4.

The drawing groups signal wires for clarity. Use the exact connection tables during assembly. The sensor and laser can share a tilt bracket mounted on a servo above the stepper's pan platform.

2. Components and their roles

The component descriptions below explain the assumed hardware. Pin assignments and operating settings throughout this guide come directly from the supplied sketch; numbered references identify the hardware documentation.

Component	Purpose and operation
Arduino Uno R3	ATmega328P controller. Reads the joystick and button, measures echo time, commands both motors, switches the laser control input and updates the LCD. [1]
Two-axis joystick	Two variable voltage outputs represent X and Y deflection. X controls pan speed; Y controls the rate at which the tilt command changes.
Pushbutton / joystick SW	Normally open switch between D2 and GND. INPUT_PULLUP makes release HIGH and press LOW. The accepted press operates the laser and built-in LED. [6]
HC-SR04 ultrasonic sensor	Emits a burst of ultrasound and reports the round-trip travel time as an ECHO pulse. Nominal sensor range: 2–400 cm. [2]
ULN2003 driver board	Transistor switches control current through the stepper coils. The ULN2003A includes clamp diodes for inductive loads. [3]
28BYJ-48 stepper motor	Geared, four-phase motor used for horizontal pan. The assumed version is rated for 5 V. It plugs into the ULN2003 board. [4]
Hobby servo	Moves the tilt bracket according to signal pulse width. The sketch commands 30°–150°, starting at 90°. Mechanical travel depends on the servo and calibration. [5]
Laser module and suitable driver	Provides a visible aiming point. D3 drives a compatible active-HIGH control input; laser power comes from its rated supply. A bare diode needs a current-regulated driver.
16×2 LCD	HD44780-compatible display using a 4-bit parallel interface. Shows distance, commanded tilt angle and button state. [7]
10 kΩ contrast potentiometer	Variable voltage divider for the LCD V0 pin. Adjust it until the characters are clear. [7]
Power supply and wiring	An external regulated supply powers the motors at their rated voltage. All ground references connect together. Choose sufficient current for motor startup and servo stall. [5]

3. Exact Arduino signal connections

Wire to the labelled Arduino headers. Preserve the IN1–IN4 order, since it must match the half-step sequence in the program.

Arduino pin	Connect to	Function
A0	Joystick VRx	Horizontal speed command
A1	Joystick VRy	Tilt rate command
D2	Pushbutton / joystick SW	Switch closes to GND
D3	Laser driver IN / TTL	Active-HIGH control
D4	ULN2003 IN1	Coil sequence input 1
D5	ULN2003 IN2	Coil sequence input 2
D6	ULN2003 IN3	Coil sequence input 3
D7	ULN2003 IN4	Coil sequence input 4
D8	HC-SR04 TRIG	Trigger pulse output
D9	HC-SR04 ECHO	Pin-change interrupt input
D10	Servo signal	Tilt command pulses
D11	LCD RS, pin 4	LCD register select
D12	LCD E, pin 6	LCD enable
A2	LCD D4, pin 11	LCD data bit 4
A3	LCD D5, pin 12	LCD data bit 5
A4	LCD D6, pin 13	LCD data bit 6
A5	LCD D7, pin 14	LCD data bit 7
D13	Built-in LED	Already connected on Uno
D0 / D1	USB serial interface	Keep available for Serial Monitor

D9 is used as a digital echo input even though it also has a PWM function on the Uno. A2–A5 are used as digital LCD outputs. In this sketch A4 and A5 do not form an I²C display connection.

4. LCD and power wiring

4.1 Parallel 16×2 LCD

```
LiquidCrystal lcd(11, 12, A2, A3, A4, A5);
//                RS   E   D4  D5  D6  D7
```

LCD pin	Label	Connection
1	VSS	Common GND
2	VDD / VCC	Arduino 5 V
3	V0 / contrast	10 kΩ potentiometer middle terminal
4	RS	Arduino D11
5	R/W	Common GND
6	E / Enable	Arduino D12
7–10	D0–D3	Leave unconnected
11	D4	Arduino A2
12	D5	Arduino A3
13	D6	Arduino A4
14	D7	Arduino A5
15	LED A / +	Backlight supply with the current limiting required by the LCD
16	LED K / −	Common GND

Connect the two outer terminals of the 10 kΩ contrast potentiometer to 5 V and GND. Its middle terminal goes to V0, LCD pin 3. LCD pins D0–D3 are unused because the interface sends each byte in two 4-bit transfers. Confirm the backlight current-limiting arrangement on your LCD module. [7]

4.2 Power distribution

Supply / connection	Loads or destination
USB	Arduino Uno
Arduino 5 V	Joystick VCC, HC-SR04 VCC, LCD VCC and one outer contrast-pot terminal
External regulated 5 V	ULN2003 motor-power + and servo +, assuming both motors are rated for 5 V

Laser-rated supply	Laser driver/module power input
Common GND	Uno GND, joystick, HC-SR04, LCD, servo, ULN2003 and every supply negative

Size the external supply for the combined motor current, including servo stall current. With the Uno powered by USB, keep the external positive motor rail separate from the Uno's 5 V rail and join their grounds. The motor current should return to the supply rather than pass through the Arduino board. [5]

D3 is a control signal. Connect it only to a suitable driver input. A bare laser diode requires a current-regulated driver; a two-wire laser module needs a correctly rated switching circuit. The sketch alone cannot identify the laser module's voltage, current or input compatibility.

5. How the program works

5.1 Startup and joystick calibration

At startup the LCD displays “Centre joystick”. The Arduino averages 32 readings from each axis to establish the resting centre, taking roughly 160 ms plus conversion time. Keep the joystick released during this period. If the computed centre is outside 300–700 ADC counts, the sketch falls back to 512.

The servo is commanded to 90°. The current stepper location becomes software position zero, and a coil pattern is energized to hold it. This does not perform a physical homing operation: centre the platform before power-up.

5.2 Horizontal movement: X-axis to pan

readAxis() subtracts the calibrated centre and ignores an area of ±70 ADC counts. Outside this dead zone it produces a signed command between −100 and +100. The sign selects direction and the magnitude requests speed.

updateStepper() converts the X command to a target rate of up to 650 half-steps per second. During starts and reversals, the rate changes in increments of 15 half-steps/s about every 10 ms. Returning the joystick to centre sets the rate to zero immediately.

The eight rows in HALF_STEP alternate single-coil and two-coil patterns. D4–D7 send those patterns to the ULN2003, which switches motor-coil current. The sequence index advances for one direction and runs backward for the other. Coils remain energized when stationary to hold position.

The software limits pan to ±1024 counted half-steps from startup zero. With approximately 4096 half-steps per output revolution, this is roughly ±90°. Actual gearbox ratio and missed steps affect the mechanical angle; the sketch has no encoder or limit-switch feedback. [4]

5.3 Vertical movement: Y-axis to tilt

Every 20 ms the program adds yCommand × 0.009 to tiltAngle. A full command therefore changes the requested angle by 0.9° per update, or nominally 45°/s. Holding the joystick off-centre keeps moving the command; releasing it holds the current angle.

The command is constrained to 30°–150° and rounded before being sent to the Servo library. attach(SERVO_PIN, 1000, 2000) maps library angle commands onto that pulse-width range. The physical angle depends on the servo and mounting. The LCD reports the commanded angle, not a measured shaft angle. [5]

5.4 Ultrasonic distance measurement

About every 100 ms, D8 provides a 10 µs HIGH trigger pulse. The HC-SR04 emits eight cycles of 40 kHz ultrasound. Its ECHO pulse represents the sound's round-trip travel time. The division by two in the physical distance equation accounts for travel to the object and back. [2]

```
distance_cm ≈ echo_pulse_width_us / 58
```

```
Example: 5800 µs / 58 ≈ 100 cm
```

The sketch accepts echo widths from 116 to 23,200 µs, corresponding approximately to 2–400 cm, and rounds the result to whole centimetres. Temperature, target angle and surface reflectivity can affect practical measurements. [2]

D9 is PB1/PCINT1 on the Uno. The PCINT0 interrupt routine records the rising edge, then subtracts that timestamp from the falling-edge timestamp. Atomic blocks protect shared 32-bit values on the 8-bit processor. Echo capture uses pin-change interrupts, leaving Timer1 available to the Servo library. [1, 8]

While the echo is pending, the main loop continues updating the controls, motors and LCD. If no complete pulse arrives within about 30 ms, the program reports NO_ECHO or ECHO_STUCK_HIGH. A completed pulse outside the accepted width range produces OUT_OF_RANGE.

5.5 Button and laser operation

The button uses INPUT_PULLUP, so an open switch reads HIGH and a press reads LOW. A press must remain stable for about 25 ms before acceptance. Once accepted, the built-in LED lights and D3 goes HIGH, provided laser output is enabled and sensor-only mode is off. [6]

Releasing the switch clears the accepted button state on the next update. After reset, the button must first be released long enough to become ready. The laser operates while the button is held; it is not automatically controlled by the measured distance.

5.6 LCD output and serial diagnostics

The first LCD row shows distance or a fault message. The second shows commanded tilt and accepted button state, for example "Tilt: 90 B:1". B:1 means an accepted press; B:0 means released. The display writes one character roughly every 3 ms to reduce interruptions to motor servicing.

Open Serial Monitor at 115200 baud. The sketch prints echo width and distance, or one of NO_ECHO, ECHO_STUCK_HIGH and OUT_OF_RANGE. LCD equivalents are "No echo", "Echo stuck HIGH" and "Out of range".

6. Software reference and setup

6.1 Function map

Function / routine	Role
setup()	Initializes interfaces, enables echo interrupts, calibrates joystick centre and positions the servo.
readAxis()	Converts an analog axis reading to a signed command from −100 to +100 with a dead zone.
updateButton()	Debounces the button and writes the laser-control and built-in LED states.
updateRange()	Schedules triggers, handles echo-ready results and detects timeouts.
ISR(PCINT0_vect)	Records ECHO rising and falling edges on D9.
updateStepper()	Ramps speed, applies travel limits and advances the half-step sequence.
updateControlsAndTilt()	Samples joystick axes and updates the servo angle command.
updateDisplay()	Formats distance/status text and writes the LCD incrementally.
reportRange()	Prints distance or fault information at 115200 baud.
loop()	Calls the update functions repeatedly so the tasks progress together.

6.2 Main settings

Setting	Value in your sketch	Effect
ENABLE_LASER_OUTPUT	true	Allows laser control when the button is held
SENSOR_ONLY_TEST	false	Motors and laser are enabled for normal operation
REVERSE_X / REVERSE_Y	false / false	Change the respective value to reverse an axis
DEAD_ZONE	70 ADC counts	Suppresses movement near joystick centre
MAX_STEP_RATE	650 half-steps/s	Maximum requested pan stepping rate
PAN_LIMIT	±1024 half-steps	Software travel from startup zero; roughly ±90°
TILT_MIN / TILT_MAX	30° / 150°	Limits the commanded tilt angle
Servo attach range	1000–2000 µs	Pulse mapping used for angle commands
Ping interval / timeout	100 ms / 30 ms	Measurement scheduling and fault detection

Joystick / tilt updates	10 ms / 20 ms	Control sampling and servo-command updates
Serial Monitor	115200 baud	Distance and error diagnostics

6.3 Assembly and first test

1. Connect the logic wiring, LCD and sensor according to the tables. Set SENSOR_ONLY_TEST = true for an initial sensor/LCD check.
2. Upload with the Arduino Uno / ATmega328P board selected. Leave the joystick centred during startup, adjust LCD contrast and open Serial Monitor at 115200 baud.
3. Present a flat target at a known distance within the sensor's nominal range and compare it with the displayed reading.
4. Connect the external motor supply and common ground. Centre the pan platform and allow free servo travel, then set SENSOR_ONLY_TEST = false and upload again.
5. Test pan and tilt slowly. Reverse an axis with REVERSE_X or REVERSE_Y if needed. Reduce MAX_STEP_RATE if the stepper stalls or misses steps.
6. Connect and test the laser control only after confirming the actual driver's supply and control-input ratings. Check that release switches it off.

Recommended mounting: the stepper rotates the base; the servo sits on that base and tilts a bracket carrying the ultrasonic sensor and laser. Their pointing directions can be aligned mechanically, but the ultrasonic sensor measures over a wider acoustic beam than the visible laser spot.

Scope and limits: this sketch provides manual aiming and continuous distance measurement. It does not implement automatic target tracking or automatic scanning. Pan position is estimated from commanded steps; resetting the board establishes a new zero. Sensor-only mode is a software test setting and does not remove supply power from the hardware.

7. Sources and technical references

Code-specific pin numbers, timing thresholds and control behavior were read from the sketch supplied in this conversation. Hardware assumptions should be checked against the markings and datasheets of the actual modules. The links below are manufacturer documentation or original datasheets hosted by distributors; accessed 20 October 2026.

[1] Arduino — UNO R3 documentation and pinout

https://docs.arduino.cc/hardware/uno-rev3/

[2] ELECFREAKS — HC-SR04 ultrasonic ranging module datasheet

https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[3] Texas Instruments — ULN2003A transistor-array datasheet

https://www.ti.com/lit/ds/symlink/uln2003a.pdf

[4] 28BYJ-48 5 V stepper motor datasheet, hosted by Mouser

https://www.mouser.com/datasheet/2/758/stepd-01-data-sheet-1143075.pdf

[5] Arduino — Servo Motor Basics with Arduino

https://docs.arduino.cc/learn/electronics/servo-motors/

[6] Arduino — Digital Input Pull-Up tutorial

https://github.com/arduino/docs-content/blob/main/content/tutorials/generic/digital-input-pullup/digital-input-pullup.md

[7] Arduino — Official LiquidCrystal HelloWorld example

https://github.com/arduino-libraries/LiquidCrystal/blob/master/examples/HelloWorld/HelloWorld.ino

[8] Arduino — Servo library timer selection source

https://github.com/arduino-libraries/Servo/blob/master/src/avr/ServoTimers.h